API Flask para regresión polinómica

Caso demostrativo: expedientes de terrazas en Madrid

IFCD107 – Especialista en Inteligencia Artificial
Proyecto práctico de API y aprendizaje automático

Octubre de 2026

Resumen

Este proyecto ofrece una API REST en Flask para ajustar una regresión polinómica por mínimos cuadrados. La capa HTTP valida la solicitud y organiza las ejecuciones por referencia; la clase `PolynomialRegressor` encapsula el algoritmo y la generación de predicciones y gráficas. Se incluyen solicitudes de Postman para JSON y carga de CSV.

1. Descripción de la solución

La aplicación recibe observaciones (x, y) y un grado polinómico de 1 a 10. El ejemplo de dominio representa la gestión de expedientes de terrazas: x es un índice didáctico de complejidad y y son días de tramitación simulados. El conjunto de 1.000 filas es sintético, no contiene expedientes reales ni reproduce estadísticas o plazos del Ayuntamiento de Madrid. La clase de regresión ajusta el modelo usando `numpy.polynomial.Polynomial.fit`, calcula predicciones, residuos y métricas R^2, RMSE y MAE. La API guarda datos de entrada, CSV de predicciones, gráfico, modelo entrenado y log por ejecución.

Componente	Responsabilidad
`app.py`	Endpoints HTTP, validación, carpetas, persistencia y respuesta JS
`regression/polynomial_regressor.py`	Ajuste matemático, métricas, CSV de salida y gráfica.
`scripts/generate_terrace_cases.py`	Generador determinista de las 1.000 filas sintéticas.
`ejemplos/datos_ejemplo.csv`	Expedientes demostrativos con contexto y campos x, y.
`postman/`	Colección importable con health, solicitud JSON y carga de CSV.
`runs/`	Artefactos generados, separados por referencia y ejecución.

2. Estructura de directorios

```
proyecto_regresion_polinomica/
  app.py
  regression/polynomial_regressor.py
  ejemplos/datos_ejemplo.csv
  postman/Regresion_Polinomica.postman_collection.json
  documentacion/guia_proyecto.tex
  runs/<referencia>/
    graficas/<run_id>_ajuste.png
    csv_entrada/<run_id>.csv
    logs_salida/<run_id>.log
    csv_salida/<run_id>_predicciones.csv
    modelos/<run_id>_modelo.pkt
```

La referencia admite letras, números, punto, guion y guion bajo, con máximo de 64 caracteres. Se rechazan rutas o separadores para evitar escapar del directorio de ejecución. Cada petición genera un identificador de ejecución único, por lo que conserva varias ejecuciones dentro de la misma referencia.

3. Instalación y arranque

Se recomienda Python 3.10 o superior. Desde la carpeta raíz del proyecto:

```
python -m venv .venv
```

Activar el entorno: en Windows PowerShell, `.venv
Scripts
Activate.ps1`; en macOS/Linux, `source .venv/bin/activate`. Después:

```
python -m pip install -r requirements.txt
python app.py
```

La API queda disponible localmente en http://127.0.0.1:5000. Para comprobar disponibilidad, abrir http://127.0.0.1:5000/api/v1/health; la respuesta esperada es JSON con `"status":.°k"`.

4. Prueba desde Postman

1. En Postman, importar `postman/Regresion_Polinomica.postman_collection.json`.
2. Mantener `baseUrl=http://127.0.0.1:5000` y usar la referencia `terrazas_madrid_demo`.
3. Ejecutar primero la petición **Health**; debe responder HTTP 200.
4. Ejecutar **Ajustar desde JSON**. La colección ya incluye datos sintéticos y grado 2. Postman establece `Content-Type: application/json`.
5. Para el caso de terrazas, ejecutar **Ajustar desde CSV**; en el campo `file`, seleccionar `ejemplos/datos_ejemplo.csv`. El CSV tiene 1.000 filas de datos, campos de referencia sintéticos y encabezados `x,y`. Postman construye multipart/form-data; no fijar manualmente el encabezado boundary.
6. Leer `artifacts` en la respuesta JSON. Las rutas son relativas al directorio del proyecto y están agrupadas bajo `runs/<reference>/`.

4.1. Solicitud JSON

```
POST /api/v1/regresion/polinomica
Content-Type: application/json

{
  "reference": "expediente_demo_001",
  "degree": 2,
  "data": [
    {"x": -1, "y": 2.2}, {"x": 0, "y": 1.0},
    {"x": 1, "y": 1.8}, {"x": 2, "y": 6.2}
  ]
}
```

4.2. Solicitud CSV

```
POST /api/v1/regresion/polinomica/csv
Body: form-data
  reference = terrazas_madrid_demo
  degree = 2
  file = ejemplos/datos_ejemplo.csv (1.000 filas, x=índice de complejidad, y=días
      simulados)
```

5. Respuesta y archivos

Una ejecución correcta responde HTTP 200 con `reference`, `run_id`, grado, número de observaciones, coeficientes en orden descendente, métricas y rutas de artefactos. El archivo `modelos/<run_id>_modelo.pk` contiene el objeto entrenado y se puede recargar con `PolynomialRegressor.load(ruta)` para llamar a `predict(x)`. El CSV de salida contiene `x,y_observado,y_predicho,residuo`. La gráfica muestra datos observados y curva ajustada. Los logs permiten consultar el resultado y errores técnicos sin devolver detalles internos en fallos inesperados.

El formato `.pkt` de este ejemplo utiliza serialización Python pickle. Solo se deben cargar archivos generados localmente y de confianza: pickle puede ejecutar código al cargar un fichero manipulado. Para intercambio entre sistemas, conviene guardar los coeficientes y metadatos en JSON en vez de cargar pickles externos.

6. Validaciones y límites del ejemplo

La API limita tamaño de petición, filas CSV y grado; exige números finitos, al menos dos observaciones y suficientes valores x distintos para el grado. Responde HTTP 400 ante entrada no válida, 413 ante contenido demasiado grande y 422 cuando no se puede ajustar el modelo. No incorpora autenticación, autorización, base de datos, colas de trabajo ni almacenamiento remoto; antes de exponerla fuera de localhost se deben añadir controles de seguridad, gestión de secretos, retención y observabilidad acordes al entorno.

7. Notas sobre interpretación

La regresión polinómica es sensible al grado, al rango y escala de x, a valores atípicos y a extrapolación. R^2 y errores se calculan sobre los mismos datos usados para el ajuste; por sí solos no estiman generalización. Para uso predictivo real, separar entrenamiento y evaluación, comparar grados con validación y revisar residuos y supuestos del dominio.

Este caso sirve para demostrar el flujo técnico, no para estimar plazos reales de expedientes ni evaluar el desempeño de unidades municipales. Para ello harían falta datos autorizados, definición institucional del inicio/fin de tramitación, tratamiento de expedientes abiertos/censurados, análisis de estacionalidad y validación temporal fuera de muestra. Se puede regenerar el fichero con `python scripts/generate_terrace_cases.py`; la semilla fija mantiene la demo reproducible.

A. Código fuente completo de la API Flask

Este listado corresponde a `app.py`, incluido también como archivo ejecutable en la raíz del proyecto. La lógica matemática se mantiene en la clase separada `regression/polynomial_regressor.py`.

Listing 1: Código fuente completo de app.py

```
"""API Flask de ejemplo para ajuste de regresión polinómica."""
from __future__ import annotations
```

```
import csv
import io
import logging
import re
from datetime import datetime, timezone
from pathlib import Path
from uuid import uuid4

import numpy as np
from flask import Flask, jsonify, request
from werkzeug.exceptions import RequestEntityTooLarge

from regression.polynomial_regressor import PolynomialRegressor

# Todas las salidas quedan ancladas al proyecto, no al directorio desde el que
# se haya lanzado Flask. Esto evita que un cambio de working directory disperse
# artefactos de una misma ejecución.
OUTPUT_ROOT = Path(__file__).resolve().parent / "runs"
# reference forma parte de una ruta de disco: la lista permitida impide `../`,
# separadores y caracteres que puedan escapar o crear rutas ambiguas.
REFERENCE_RE = re.compile(r"^[A-Za-z0-9][A-Za-z0-9._-]{0,63}$")
# Límite lógico independiente del límite de bytes: protege memoria y coste del
# ajuste incluso si el cliente comprime o envía valores muy cortos.
MAX_POINTS = 100_000


def create_app(output_root: Path = OUTPUT_ROOT) -> Flask:
    app = Flask(__name__)
    # Flask rechaza cuerpos excesivos antes de parsearlos; el endpoint CSV aplica
    # además un máximo menor sobre el archivo concreto.
    app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
    output_root = Path(output_root)

    @app.errorhandler(RequestEntityTooLarge)
    def too_large(_error):
        return jsonify(error="El cuerpo supera el límite de 5 MB."), 413

    @app.get("/api/v1/health")
    def health():
        return jsonify(status="ok", service="regresion-polinomica-api", version="1.0.0")

    @app.post("/api/v1/regresion/polinomica")
    def fit_json():
        # `silent=True` permite devolver un 400 uniforme para JSON mal formado,
        # en lugar de que el detalle interno del parser llegue al cliente.
        body = request.get_json(silent=True)
        if not isinstance(body, dict):
            return jsonify(error="El cuerpo debe ser un objeto JSON."), 400
        reference, error = validate_reference(body.get("reference"))
        if error:
            return jsonify(error=error), 400
        # El grado controla la complejidad y estabilidad del ajuste. Se limita
        # antes de construir el modelo para evitar modelos excesivamente flexibles.
        degree = body.get("degree", 2)
        if isinstance(degree, bool) or not isinstance(degree, int) or degree < 1 or degree
             > 10:
            return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
        points = body.get("data")
        try:
            x, y = parse_json_points(points)
```

```
        except ValueError as exc:
            return jsonify(error=str(exc)), 400
        return run_model(app, output_root, reference, degree, x, y, "json")

    @app.post("/api/v1/regresion/polinomica/csv")
    def fit_csv():
        # Multipart separa los parámetros de control del fichero binario y es el
        # formato que Postman usa para adjuntar CSV sin codificarlo en JSON.
        reference, error = validate_reference(request.form.get("reference"))
        if error:
            return jsonify(error=error), 400
        try:
            degree = int(request.form.get("degree", "2"))
        except (TypeError, ValueError):
            return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
        if not 1 <= degree <= 10:
            return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
        uploaded = request.files.get("file")
        if uploaded is None or not uploaded.filename:
            return jsonify(error="Falta el archivo CSV en el campo multipart 'file'."), 400
        # Se lee una sola vez y se controla el tamaño antes de decodificarlo; luego
        # se conserva el original recibido como rastro reproducible de la entrada.
        raw = uploaded.read()
        if len(raw) > 4 * 1024 * 1024:
            return jsonify(error="El CSV supera el límite de 4 MB."), 413
        try:
            text = raw.decode("utf-8-sig")
            x, y = parse_csv(text)
        except (UnicodeDecodeError, ValueError) as exc:
            return jsonify(error=f"CSV no válido: {exc}"), 400
        return run_model(app, output_root, reference, degree, x, y, "csv", raw_csv=raw)

    return app


def validate_reference(value):
    # La validación se aplica antes de formar cualquier ruta. No se normaliza una
    # referencia inválida porque eso podría hacer colisionar expedientes distintos.
    if not isinstance(value, str) or not REFERENCE_RE.fullmatch(value):
        return None, "reference es obligatorio y debe tener 1-64 caracteres: letras, nú
            meros, punto, guion o guion bajo."
    return value, None


def parse_json_points(points):
    # Limitar número de filas antes de convertirlas a arrays evita asignaciones
    # grandes no necesarias; después se comprueba tipo y finitud en validate_xy.
    if not isinstance(points, list) or not 2 <= len(points) <= MAX_POINTS:
        raise ValueError(f"data debe contener entre 2 y {MAX_POINTS} pares x/y.")
    try:
        x = np.asarray([row["x"] for row in points], dtype=float)
        y = np.asarray([row["y"] for row in points], dtype=float)
    except (TypeError, ValueError, KeyError, IndexError):
        raise ValueError("Cada elemento de data debe tener valores numéricos x e y.") from
             None
    return validate_xy(x, y)


def parse_csv(text):
    # DictReader permite conservar columnas contextuales del caso de terrazas,
```

```
    # aunque el algoritmo solo use x e y. Las filas extra no se descartan del
    # CSV archivado; solo se extraen aquí las dos variables del modelo.
    reader = csv.DictReader(io.StringIO(text))
    if not reader.fieldnames or not {"x", "y"}.issubset(set(reader.fieldnames)):
        raise ValueError("El encabezado debe contener las columnas x,y.")
    x_values, y_values = [], []
    for line_number, row in enumerate(reader, start=2):
        try:
            x_values.append(float(row["x"]))
            y_values.append(float(row["y"]))
        except (TypeError, ValueError):
            raise ValueError(f"Valores no numéricos en la línea {line_number}.") from None
        if len(x_values) > MAX_POINTS:
            raise ValueError(f"El CSV supera {MAX_POINTS} filas de datos.")
    return validate_xy(np.asarray(x_values), np.asarray(y_values))


def validate_xy(x, y):
    # NaN e infinito contaminan coeficientes y métricas sin producir siempre una
    # excepción clara; se rechazan en el borde HTTP antes de invocar NumPy.
    if x.ndim != 1 or y.ndim != 1 or len(x) != len(y) or len(x) < 2:
        raise ValueError("Se requieren al menos dos pares x/y del mismo tamaño.")
    if not np.isfinite(x).all() or not np.isfinite(y).all():
        raise ValueError("x e y solo pueden contener números finitos.")
    return x, y


def run_model(app, output_root, reference, degree, x, y, input_kind, raw_csv=None):
    # El sufijo aleatorio evita sobrescrituras si llegan dos peticiones para la
    # misma referencia dentro del mismo segundo. UTC facilita correlación de logs.
    run_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ") + "_" + uuid4().hex
        [:8]
    base = Path(output_root) / reference
    # Un directorio raíz por referencia y carpetas por tipo de artefacto hacen
    # posible auditar/reprocesar cada caso sin mezclar entrada, modelo y salida.
    folders = {name: base / name for name in ("graficas", "csv_entrada", "logs_salida", "
        csv_salida", "modelos")}
    for folder in folders.values():
        folder.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger(f"regression.{reference}.{run_id}")
    logger.setLevel(logging.INFO)
    handler = logging.FileHandler(folders["logs_salida"] / f"{run_id}.log", encoding="utf
        -8")
    handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
    logger.addHandler(handler)
    try:
        input_path = folders["csv_entrada"] / f"{run_id}.csv"
        if raw_csv is None:
            # Para JSON también se guarda una entrada tabular mínima; así cada run
            # tiene una fuente de datos recuperable independientemente del canal.
            with input_path.open("w", newline="", encoding="utf-8") as stream:
                writer = csv.writer(stream)
                writer.writerow(["x", "y"])
                writer.writerows(zip(x.tolist(), y.tolist()))
        else:
            # Guardar bytes originales preserva columnas extra, orden, formato y
            # encabezados exactamente como los envió el cliente.
            input_path.write_bytes(raw_csv)

        model = PolynomialRegressor(degree=degree)
```

```
        result = model.fit(x, y)
        csv_path = folders["csv_salida"] / f"{run_id}_predicciones.csv"
        graph_path = folders["graficas"] / f"{run_id}_ajuste.png"
        model_path = folders["modelos"] / f"{run_id}_modelo.pkt"
        # Persistir el modelo inmediatamente tras entrenar permite volver a usarlo
        # aunque la API se reinicie; el endpoint devuelve la ruta para trazabilidad.
        model.save(model_path)
        model.write_predictions(csv_path, result)
        model.write_plot(graph_path, x, y, result)
        metadata = {
            "reference": reference,
            "run_id": run_id,
            "degree": degree,
            "n_observations": int(len(x)),
            "coefficients_descending": result.coefficients.tolist(),
            "metrics": result.metrics,
            "artifacts": {
                "input_csv": str(input_path.relative_to(output_root)),
                "output_csv": str(csv_path.relative_to(output_root)),
                "graph_png": str(graph_path.relative_to(output_root)),
                "trained_model_pkt": str(model_path.relative_to(output_root)),
                "log": str((folders["logs_salida"] / f"{run_id}.log").relative_to(
                    output_root)),
            },
        }
        logger.info("Ajuste completado input=%s degree=%d n=%d metrics=%s", input_kind,
            degree, len(x), result.metrics)
        return jsonify(metadata), 200
    except (ValueError, np.linalg.LinAlgError) as exc:
        # Errores de datos/ajuste se registran con el run_id y reciben 422; el log
        # conserva traceback para diagnóstico sin mezclarlo con la respuesta normal.
        logger.exception("No se pudo ajustar el modelo: %s", exc)
        return jsonify(error=f"No se pudo ajustar el modelo: {exc}", reference=reference,
            run_id=run_id), 422
    except Exception as exc: # No devolver rutas, stack traces ni secretos al cliente.
        logger.exception("Error interno en el ajuste: %s", exc)
        return jsonify(error="Error interno durante el ajuste.", reference=reference,
            run_id=run_id), 500
    finally:
        # Cerrar el handler por solicitud evita descriptores abiertos durante un
        # servicio prolongado y permite leer/archivar el log al terminar la petición.
        handler.close()
        logger.removeHandler(handler)


app = create_app()

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```